Online Restaurant ordering system

User Manual

Presented By:
Cobra.py

May 3, 2026
UNCC

Online Restaurant Ordering System API - User Manual

1. Overview

The Online Restaurant Ordering System API allows users to create, view, update, and delete restaurant orders and order details. It is built using FastAPI and provides simple, predictable endpoints for interacting with order data.
This manual explains how to set up and run the API. Also, how to use each endpoint, with clear examples.

2. Setup Instructions

Follow these steps to install and run the API on your machine.

Step 1: Install Dependencies

Make sure you have Python installed. Then install the required packages:
pip install -r requirements.txt

Step 2: Configure the Database

The API uses a MySQL database. Update your database connection settings inside:
api/dependencies/database.py
Set your MySQL username, password, host, and database name.

Step 3: Start the API Server

Run the following command from the project root:
uvicorn api.main:app --reload
The API will be available at:
http://127.0.0.1:8000

Step 4: View API Documentation

FastAPI automatically generates interactive documentation:

- Swagger UI: http://127.0.0.1:8000/docs

3. How to Use the API

The API contains two main resource groups:

- Orders (/orders)
- Order Details (/orderdetails)

Each group supports creating, reading, updating, and deleting data.

4. Orders Endpoints

Base URL: /orders

Create an Order

POST /orders/
Request Body Example:

```
{
  "customer_name": "John Doe",
  "description": "2 sandwiches and 1 drink"
}
```

Response Example:

```
{
  "id": 1,
  "customer_name": "John Doe",
  "description": "2 sandwiches and 1 drink"
}
```

Get All Orders

GET /orders/
Response Example:

```
[
  {
    "id": 1,
    "customer_name": "John Doe",
    "description": "2 sandwiches and 1 drink"
  }
]
```

Get One Order

GET /orders/{item_id}
Response Example:

```
{
  "id": 1,
  "customer_name": "John Doe",
```

```
  "description": "2 sandwiches and 1 drink"
}
```

Update an Order

PUT /orders/{item_id}
Request Body Example:

```
{
  "description": "Updated order description"
}
```

Response Example:

```
{
  "id": 1,
  "customer_name": "John Doe",
  "description": "Updated order description"
}
```

Delete an Order

DELETE /orders/{item_id}
Response: Status code 204 No Content

5. Order Details Endpoints

Base URL: /orderdetails

Create Order Detail

POST /orderdetails/
Request Body Example:

```
{
  "order_id": 1,
  "sandwich_id": 3,
  "amount": 2
}
```

Response Example:

```
{
  "id": 1,
  "order_id": 1,
  "sandwich_id": 3,
  "amount": 2
}
```

Get All Order Details

GET /orderdetails/
Response Example:

```
[
  {
    "id": 1,
    "order_id": 1,
    "sandwich_id": 3,
    "amount": 2
  }
]
```

Get One Order Detail

GET /orderdetails/{item_id}
Response Example:

```
{
  "id": 1,
  "order_id": 1,
  "sandwich_id": 3,
  "amount": 2
}
```

Update Order Detail

PUT /orderdetails/{item_id}
Request Body Example:

```
{
  "amount": 5
}
```

Response Example:

```
{
  "id": 1,
  "order_id": 1,
  "sandwich_id": 3,
  "amount": 5
}
```

Delete Order Detail

DELETE /orderdetails/{item_id}
Response: Status code 204 No Content

6. Error Handling

The API returns clear error messages when something goes wrong.

Examples:

- 404 Not Found: When an item does not exist
- 400 Bad Request: When invalid data is sent

7. Summary

This API provides a simple and clean interface for managing restaurant orders and order details. With predictable endpoints and a simple setup, it is suitable for learning, prototyping, or being integrated into a larger restaurant management system.
You can explore all endpoints interactively using the built-in Swagger documentation at:
http://127.0.0.1:8000/docs